



**Министерство науки и высшего образования Российской
Федерации**
**Федеральное государственное автономное образовательное
учреждение высшего образования**
**«Московский государственный технический университет
имени Н.Э. Баумана**
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 1
по дисциплине «Анализ алгоритмов»

Тема Сравнительный анализ ресурсной эффективности алгоритмов

Студент Дудырев Д.С.

Группа ИУ7-52Б

Преподаватели Волкова Л.Л.

Преподаватели Строганов Д.М.

Москва, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Аналитическая часть	4
1.1 Поиск полным перебором	4
1.2 Бинарный поиск	4
1.3 Вывод	5
2 Конструкторская части	6
2.1 Требования к программному обеспечению	6
2.2 Разработка алгоритмов	6
2.3 Схемы алгоритмов	6
2.4 Вывод	10
3 Технологическая часть	11
3.1 Средства разработки	11
3.2 Реализация алгоритмов	11
3.3 Тестирование	14
3.4 Вывод	15
4 Исследовательская часть	16
4.1 Вывод	16
ЗАКЛЮЧЕНИЕ	17

ВВЕДЕНИЕ

Цель: Выполнить сравнительный анализ ресурсной эффективности алгоритмов

Задачи работы:

- 1) описать решаемую согласно варианту задачу;
- 2) разработать рекурсивный и нерекурсивные алгоритмы решения задачи, согласно варианту;
- 3) описать разработанные алгоритмы схемами алгоритмов;
- 4) описать требования к программному обеспечению;
- 5) описать средства реализации алгоритмов;
- 6) реализовать разработанные алгоритмы;
- 7) выполнить тестирование реализации алгоритмов;
- 8) выполнить теоретическую оценку затрачиваемой реализацией каждого алгоритма памяти;
- 9) оценить временную эффективность алгоритмов или их реализаций в лучшем и худшем случае в терминах числа сравнений искомой величины с другими величинами;
- 10) представить результаты сравнительного анализа в формате таблицы;
- 11) сделать выводы из сравнительного анализа реализации разработанных алгоритмов решения одной и той же задачи, заданной вариантом, по критериям временной и ёмкостной эффективности, а также дополнительных требований, налагаемых алгоритмом.

1 Аналитическая часть

В данной лабораторной работе рассматривается 2 алгоритма поиск элемента в массиве:

- поиск полным перебором
- бинарный поиск

1.1 Поиск полным перебором

Пусть дан массив из N различных элементов. Рассматривается каждый элемент массива и сравнивается с искомым. Если рассматриваемый элемент совпадает с искомым его индекс запоминается и считается результатом поиска. Общая сложность алгоритма $O(N)$.

1.2 Бинарный поиск

Пусть дан отсортированный по возрастанию массив *array* из N различных чисел, искомый элемент t . $l = 0, r = N - 1$ текущие индексы левой и правой границы массива соответственно. В каждой итерации поиска определяется средний элемент массива с индексом

$$m_i = l + \frac{r - l}{2} \quad (1.1)$$

Для следующей итерации определяются новые границы массива

$$l = \begin{cases} m + 1 & \text{array}[m] < t \\ l & \text{иначе} \end{cases} \quad (1.2)$$

$$r = \begin{cases} m - 1 & \text{array}[m] \geq t \\ r & \text{иначе} \end{cases} \quad (1.3)$$

Поиск продолжается до тех пор пока $l < r$

Уменьшение размера массива в 2 раза на каждой итерации позволяет добавить временной сложности $O(\log N)$, но при этом требуется чтобы массив поступал и поддерживался отсортированным

1.3 Вывод

Алгоритм бинарного поиска является более более оптимизированный методом поиска элемента в массиве, но накладывает дополнительные ограничения — поддержка массива отсортированным. При этом алгоритм полного перебора требует больше временных затрат, но при этом нет требований для входного массива

2 Конструкторская части

2.1 Требования к программному обеспечению

- 1) Программа запрашивает размер массива N — натуральное число;
- 2) Программа формирует массив из N различных целых чисел.
- 3) Программа создаёт копию исходного массива и сортирует её по возрастанию; исходный массив не изменяется.
- 4) Программа циклически выводит меню со следующими пунктами:
 - Выбранный алгоритм
 - Поиск элемента текущим алгоритмом;
 - Поиск элемента всеми алгоритмами;
 - Выбор алгоритма;
 - Вывод массива;
 - Выполнение тестов;
 - Завершение работы.

2.2 Разработка алгоритмов

Для каждого алгоритма определены:

- *array* — входной массив
- n — количество элементов в *array*
- *target* — искомых элемент
- *index* — индекс искомого элемента

Для алгоритмов бинарного поиска определены:

- l — индекс левой границы
- r — индекс правой границы
- m — индекс рассматриваемого элемента

2.3 Схемы алгоритмов

Схема алгоритма поиска полным перебором приведена на рисунке 2.1,

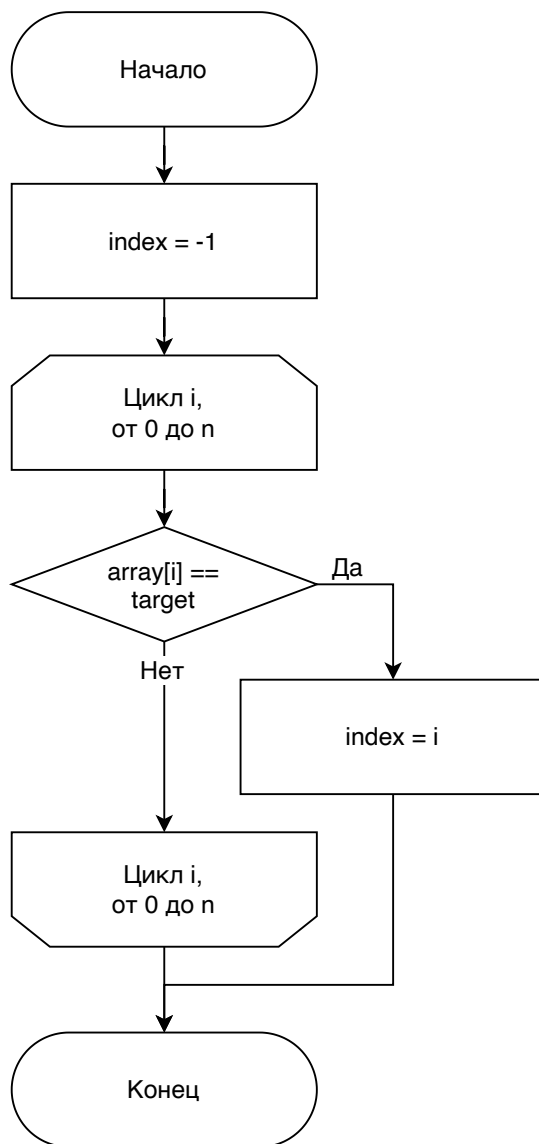


Рисунок 2.1 — Схема алгоритма поиска полным перебором

Схемы реализаций алгоритмов нерекурсивного бинарного поиска приведены на рисунках 2.2, 2.3

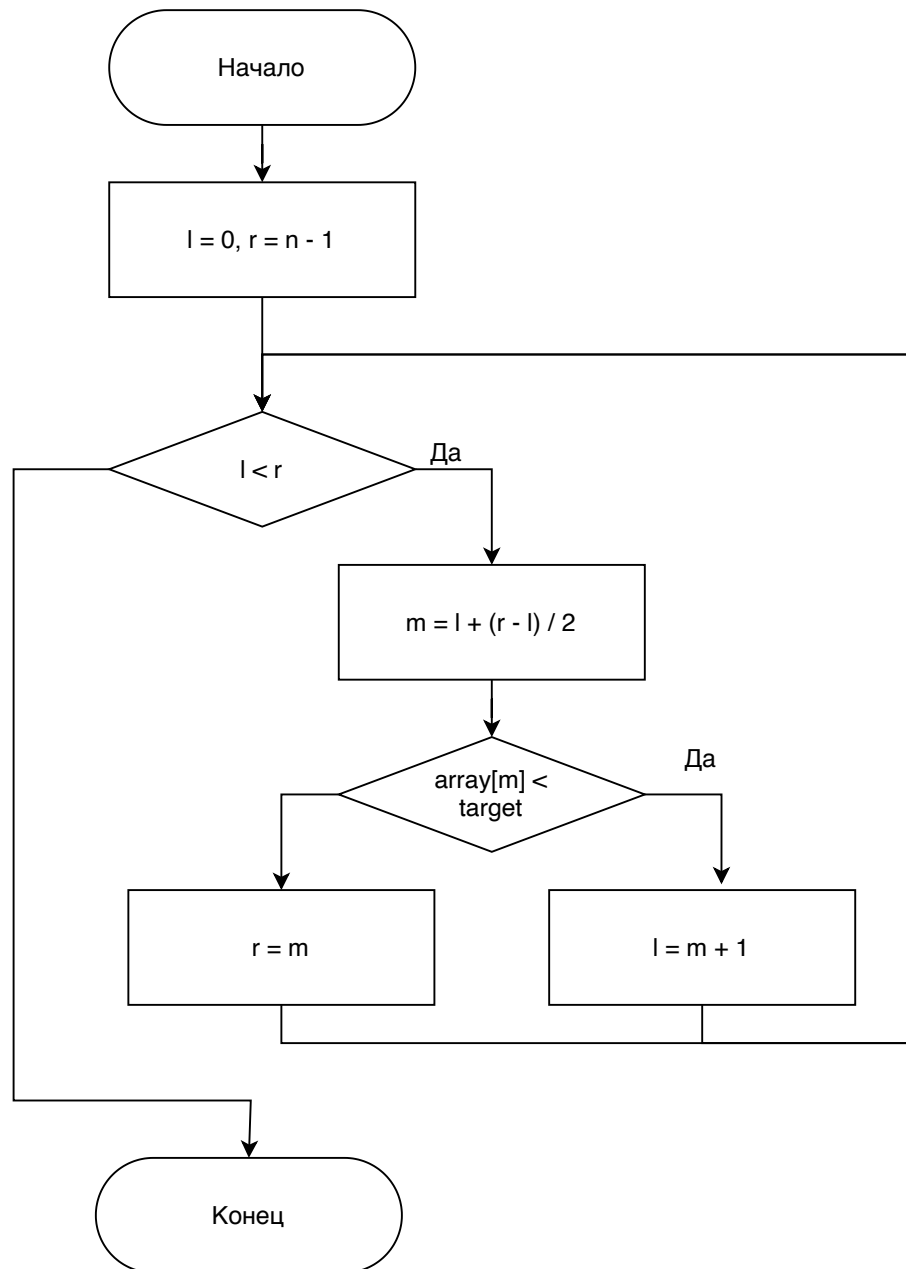


Рисунок 2.2 — Схема нерекурсивного алгоритма бинарного поиска

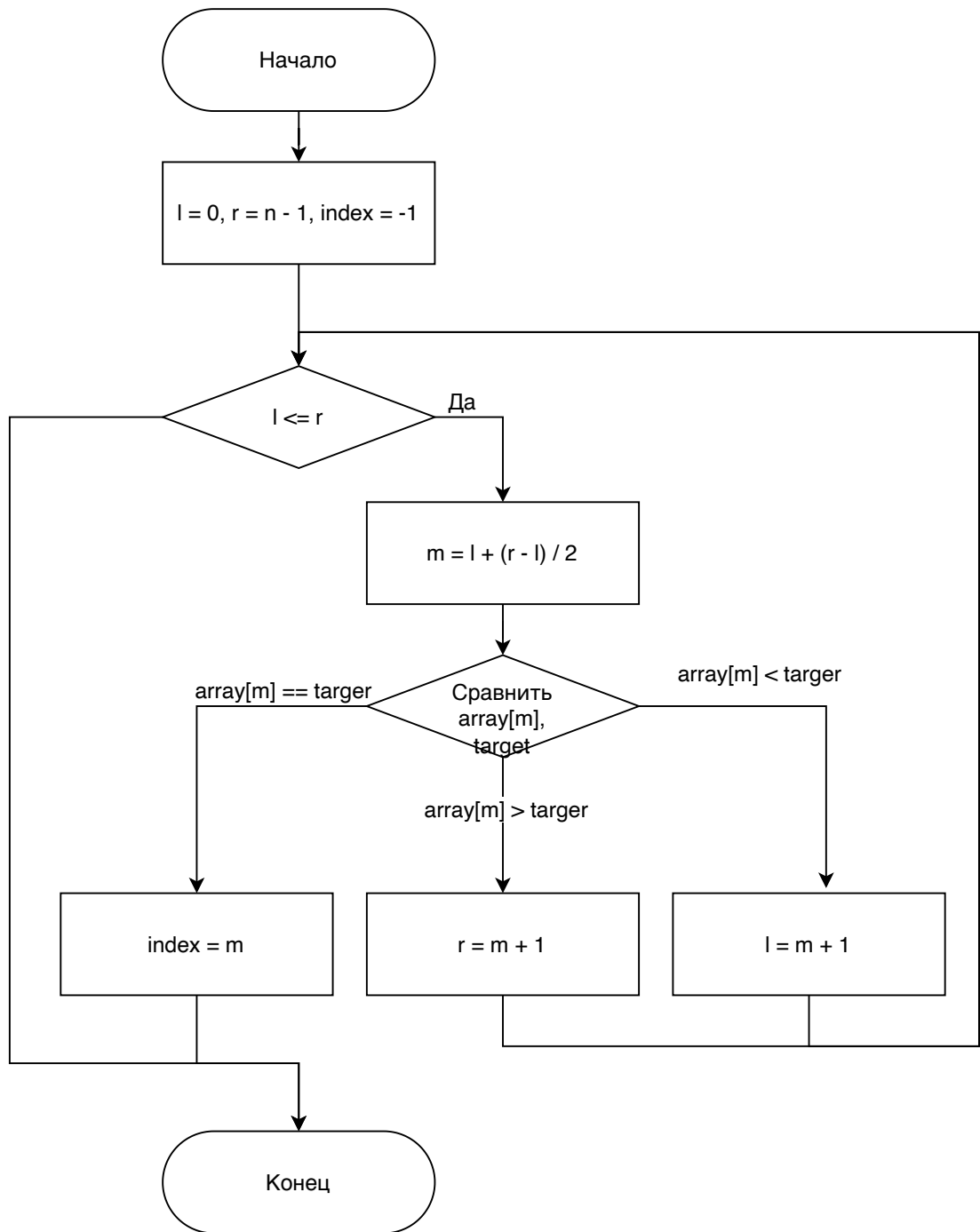


Рисунок 2.3 — Схема нерекурсивного алгоритма бинарного поиска с предварительным выходом

Схемв реализаций алгоритмов рекурсивного бинарного поиска приведены на рисунках 2.2

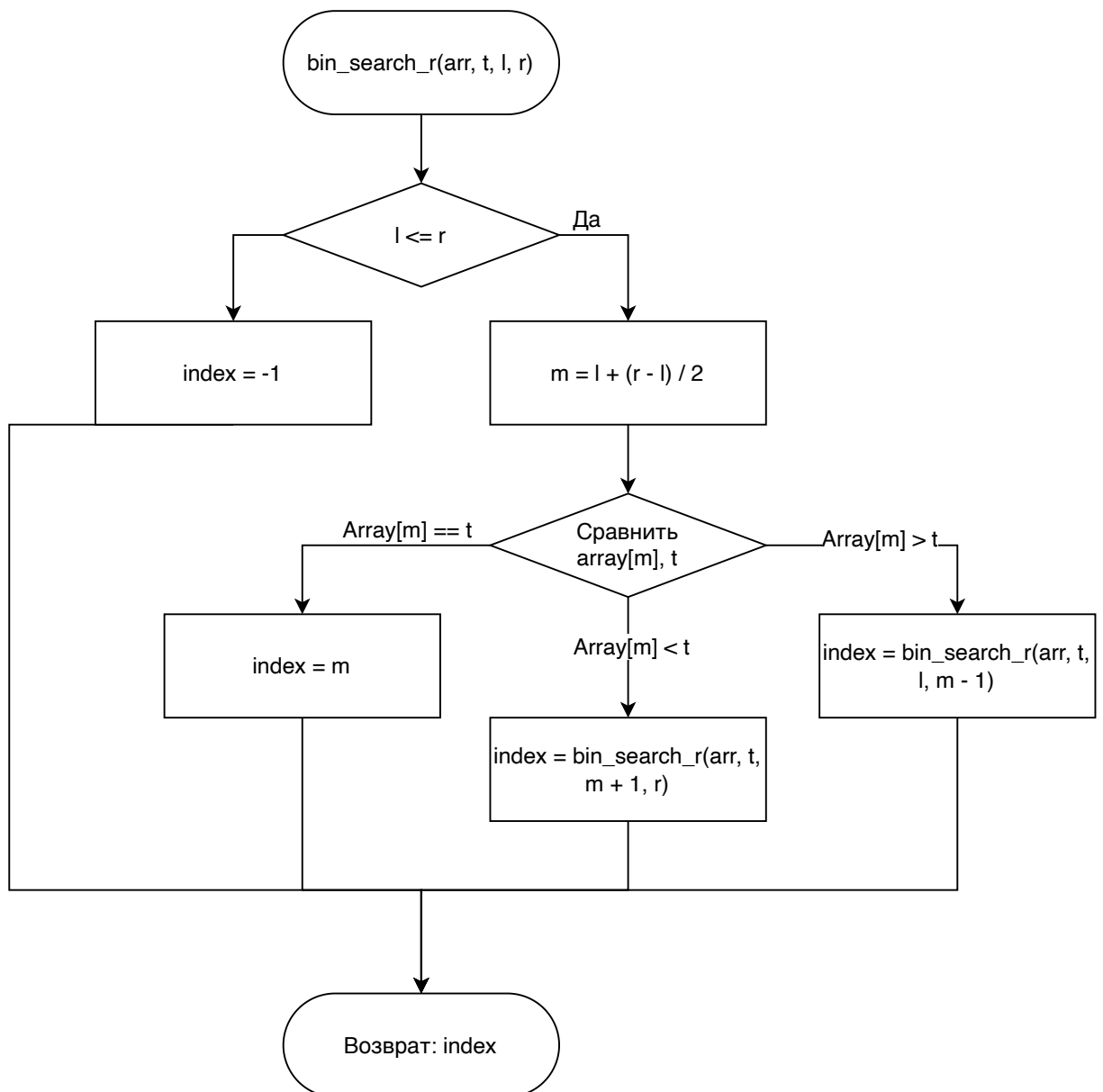


Рисунок 2.4 — Схема рекурсивного алгоритма бинарного поиска

2.4 Вывод

В результате конструкторской части были определены требования к программному обеспечению, спроектированы алгоритм полного перебора, 2 варианта алгоритма нерекурсивного бинарного поиска, алгоритм рекурсивного бинарного поиска

3 Технологическая часть

3.1 Средства разработки

В качестве языка программирования был выбран Swift, так как он является языком со статической типизацией и является основным языком для разработки на операционную систему MacOS, на которой выполнялась лабораторная работа.

3.2 Реализация алгоритмов

Листинг 3.1 — Поиск полным перебором

```
func brute_force_search(_ array: [Int], target: Int) -> Int {  
    var index = -1  
    for i in 0..  
array.count {  
        if array[i] == target {  
            index = i  
            break  
        }  
    }  
    return index  
}
```

Листинг 3.2 — Нерекурсивный бинарный поиск

```
отложенный выход то есть досхождение левой и правой границ  
func bin_search_1(_ array: [Int], target: Int) -> Int {  
    var l = 0, r = array.count - 1  
    while l < r {  
        let m = l + (r - l) / 2  
        if array[m] < target {  
            l = m + 1  
        } else {  
            r = m  
        }  
    }  
    if array[l] == target {  
        return l  
    } else {  
        return -1  
    }  
}
```

Листинг 3.3 — Нерекursивный бинарный поиск с предварительным выходом

проверка среднего элемента каждый раз

```
func bin_search_2(_ array: [Int], target: Int) -> Int {  
    var l = 0, r = array.count - 1  
    while l <= r {  
        let m = l + (r - l) / 2  
        if array[m] == target {  
            return m  
        }  
        if array[m] < target {  
            l = m + 1  
        } else {  
            r = m - 1  
        }  
    }  
  
    return -1  
}
```

Листинг 3.4 — Рекурсивный бинарный поиск

```
private func bin_search_recursion(_ array: [Int], _ target: Int, _ l: Int, _ r:  
    Int) -> Int {  
    if l > r {  
        return -1  
    }  
  
    let m = l + (r - l) / 2  
  
    if array[m] == target {  
        return m  
    } else if array[m] < target {  
        return bin_search_recursion(array, target, m + 1, r)  
    } else {  
        return bin_search_recursion(array, target, l, m - 1)  
    }  
}
```

Листинг 3.5 — Оберточная функция для рекурсивного бинарного поиска

```
func bin_search_recursion(_ array: [Int], _ target: Int) -> Int {  
    bin_search_recursion(array, target, 0, array.count - 1)  
}
```

Для единообразия интерфейсам функции при реализации рекурсивного бинарного поиска 3.4 используется оберточная функция 3.5

3.3 Тестирование

Таблица 3.1 — Функциональные тесты

№	Входные данные		Выходные данные		Комментарий
	Массив	x	Ожидаемые	Полученные	
1	[1, 2, 3, 4, 5, 6, 7, 8]	1	0	0	Первый элемент
2	[1, 2, 3, 4, 5, 6, 7, 8]	8	7	7	Последний элемент
3	[1, 2, 3, 4, 5, 6, 7, 8]	5	4	4	Середина, чётная длина
4	[1, 2, 3, 4, 5, 6, 7]	4	3	3	Середина, нечётная длина
5	[1, 2, 3, 4, 5, 6, 7, 8]	10	Не найден	Не найден	Больше максимума
6	[3, 4, 5, 6, 7, 8, 9, 10]	1	Не найден	Не найден	Меньше минимума
7	[1, 2, 3, 4, 7, 8, 9, 10]	5	Не найден	Не найден	Пропуск в середине
8	[10, 20, 30, 40]	25	Не найден	Не найден	Между соседними значениями
9	[42]	42	0	0	Один элемент, найден
10	[42]	7	Не найден	Не найден	Один элемент, не найден
11	[1, 2]	1	0	0	Два элемента, первый
12	[1, 2]	2	1	1	Два элемента, второй
13	[1, 2]	3	Не найден	Не найден	Два элемента, не найден
14	[−10, −5, −1, 0, 3]	−5	1	1	Отрицательные значения
15	[1, 100, 1000, 10 000]	1000	2	2	Разреженный массив

Все тесты пройдены успешно

3.4 Вывод

В результате технологической части были выбраны средства реализации, реализованы алгоритм поиска полным перебором, два варианта нерекурсивного бинарного поиска и рекурсивный бинарный поиск, а также проведено их функциональное тестирование: все реализации прошли тесты, приведённые в таблице 3.1.

4 Исследовательская часть

Таблица 4.1 — Сравнение алгоритмов поиска

Критерии Алгоритм	Оценка временной эффективности как функции от числа необходимых сравнений		Требования, налагаемые алгоритмом	Оценка памяти, затрачиваемой алгоритмом сверх входных данных
	ЛС	ХС		
Поиск полным перебором	1	N	Нет	$i \rightarrow O(1)$, 1 вызов функции
Нерекурсивный бинарный поиск	$1 + \log_2 N$	$1 + \log_2 N$	Массив предварительно отсортирован и поддерживается отсортированным	$l, r, m \rightarrow O(1)$, 1 вызов функции
Нерекурсивный бинарный поиск (предварительный выход)	1	$2 \log_2 N$		$\log_2 N$ вызовов, т. е. $\log_2 N \cdot (\text{размер кадра стека})$ памяти
Рекурсивный бинарный поиск	1	$2 \log_2 N$		

4.1 Вывод

Алгоритм поиска полным перебором в худшем случае выполняет N сравнений искомого элемента с элементами массива, при этом не налагает дополнительных требований на массив. Алгоритмы бинарного поиска позволяют сократить количество сравнений до логарифмического, но требуют поддержания массива в отсортированном состоянии. Использование предварительного выхода в бинарном поиске позволяет в лучшем случае сократить количество сравнений до 1, но в худшем случае число сравнений возрастает в 2 раза. Временная эффективность рекурсивной реализации совпадает с эффективностью нерекурсивной реализации с предварительным выходом, но рекурсивная реализация дополнительно затрачивает память на кадры стека.

ЗАКЛЮЧЕНИЕ

Цель работы достигнута: выполнен сравнительный анализ ресурсной эффективности алгоритмов поиска элемента в массиве — поиска полным перебором и трёх реализаций бинарного поиска.

Все задачи решены:

- 1) описана решаемая согласно варианту задача поиска элемента в массиве;
- 2) разработаны алгоритм поиска полным перебором, два нерекурсивных алгоритма бинарного поиска (без предварительного выхода и с предварительным выходом) и рекурсивный алгоритм бинарного поиска;
- 3) разработанные алгоритмы описаны схемами алгоритмов;
- 4) описаны требования к программному обеспечению;
- 5) описаны средства реализации алгоритмов, в качестве языка программирования выбран Swift;
- 6) разработанные алгоритмы реализованы;
- 7) выполнено тестирование реализаций алгоритмов, все реализации прошли функциональные тесты;
- 8) выполнена теоретическая оценка памяти, затрачиваемой реализацией каждого алгоритма;
- 9) оценена временная эффективность алгоритмов в лучшем и худшем случае в терминах числа сравнений искомой величины с другими величинами;
- 10) результаты сравнительного анализа представлены в формате таблицы;
- 11) сделаны выводы из сравнительного анализа реализаций разработанных алгоритмов по критериям временной и ёмкостной эффективности, а также дополнительных требований, налагаемых алгоритмом.

По результатам сравнительного анализа можно сделать вывод, что выбор алгоритма поиска определяется компромиссом между числом сравнений, затратами памяти и требованиями к входному массиву. Поиск полным перебором является наиболее универсальным, так как применим к произвольному массиву, однако число сравнений в худшем случае растёт линейно с размером массива. Бинарный поиск сокращает число сравнений до логарифмического, но применим только к отсортированному массиву, поэтому оправдан, когда массив уже упорядочен или поиск выполняется многократно. Среди реализаций бинарного

поиска предварительный выход выгоден в лучшем случае, но в худшем случае требует примерно вдвое больше сравнений, а рекурсивная реализация при той же временной эффективности уступает нерекурсивным по памяти из-за затрат на кадры стека.